

Teaching Exam

Subject – Computer Science

**Web Based Application
Development**



Lecture No -2

By-Satyendra Chaudhary Sir

Topics to be Covered

Topic

Client Server Architecture



Client Server Architecture

1) System Application (OS)

2) Web Application  80%

3) Mobile Applications

4) Desktop Applications (Tally, MS Office)

websites ⇒

web applications =

1. set of static webpages

Same = ?

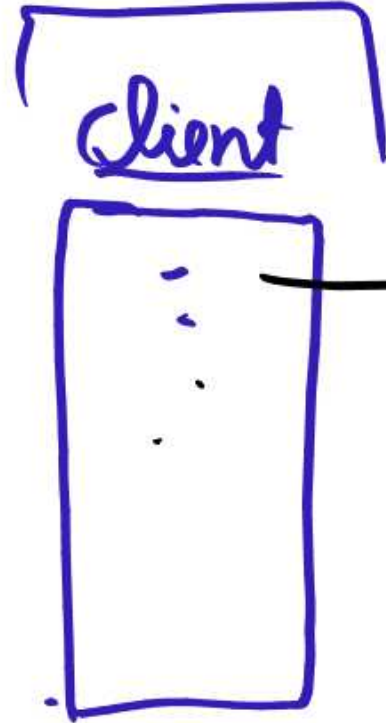
Yes / No

Glou

web applications

3-tier architecture.

Frontend

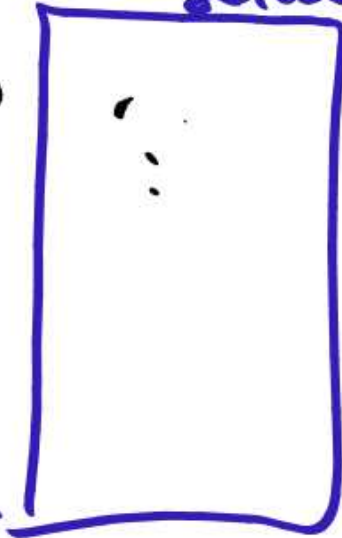


url

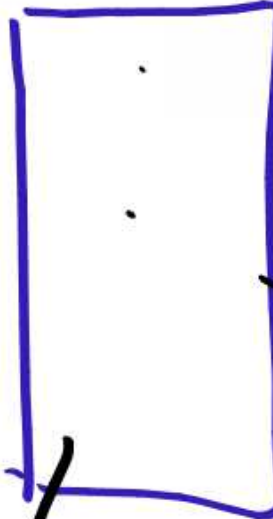


Backend.

Application.
server.



Database.
Server.



Relational. (table)

Non Relational. (Doc)

3 layers

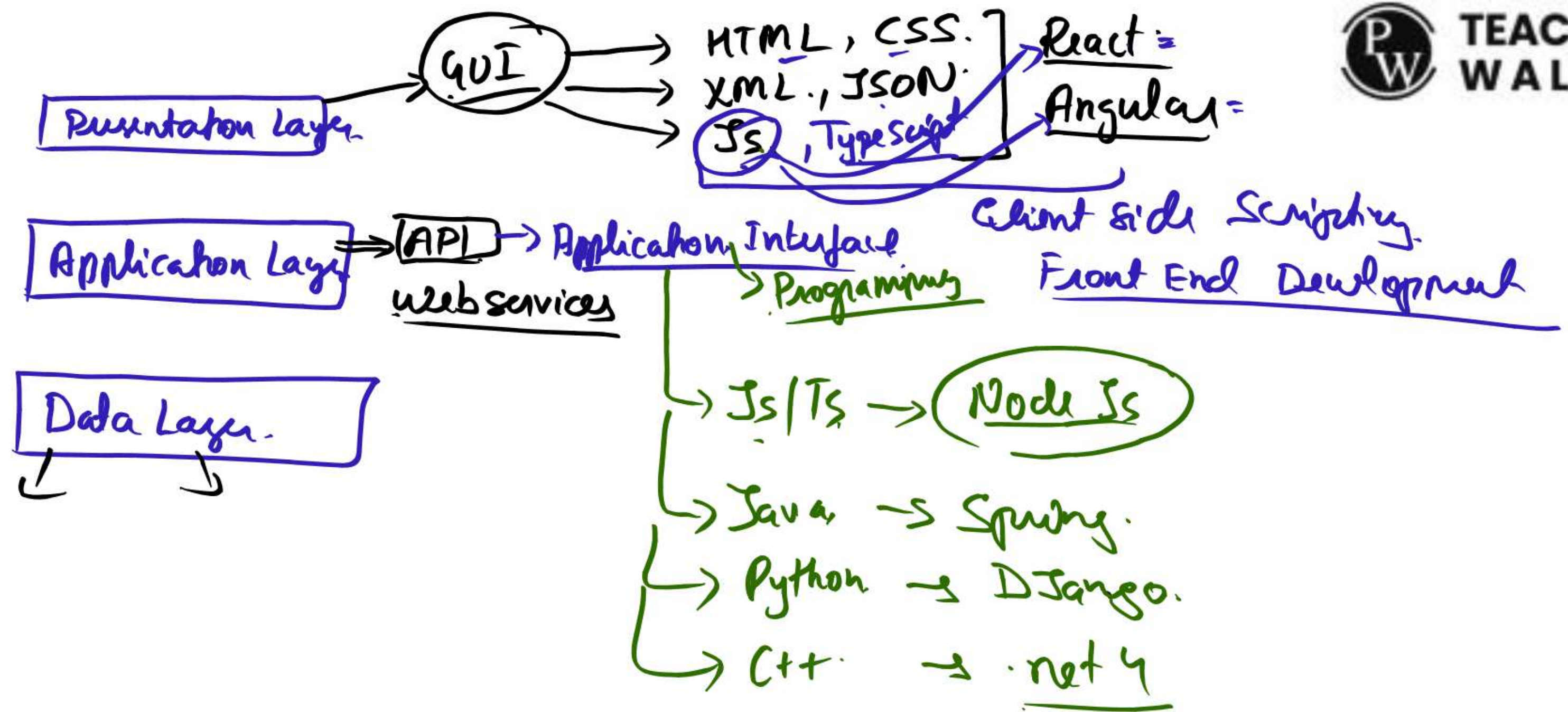
Presentation Layer.

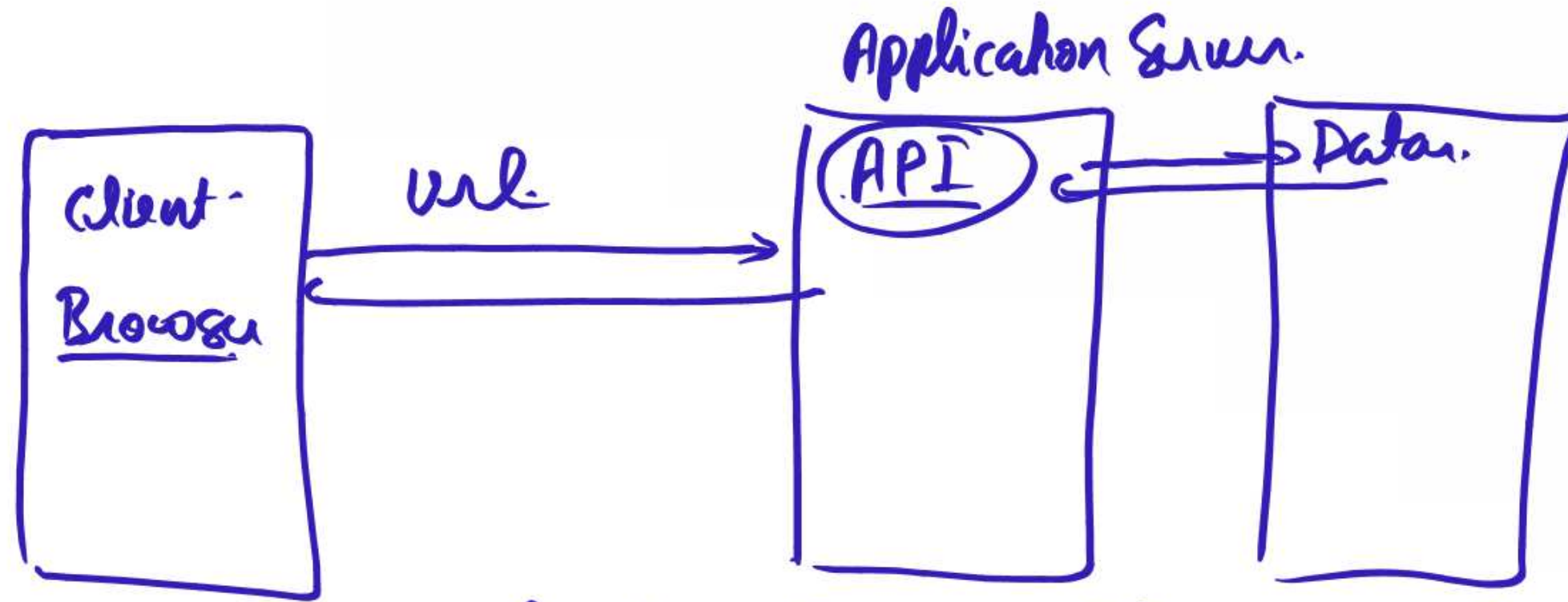
Application layer.

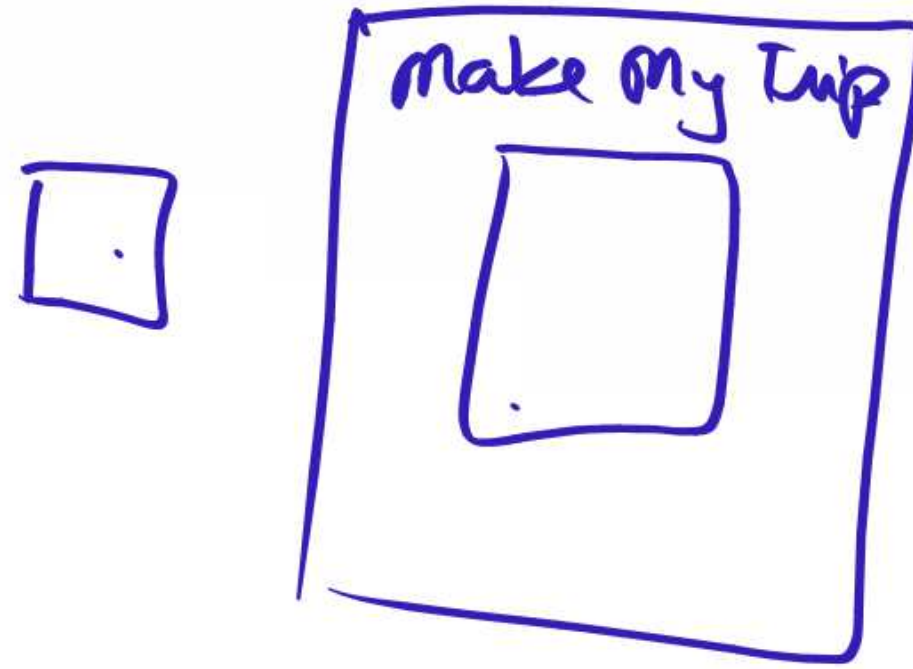
Data layer.

Mongo DB

Oracle
MS SQL







➤ India

Farm

➤ Air India

➤ Vietnam

Types of API

Simple object Access.
control

SOAP → old technology
XML.

REST → new technology.

Representational.
State-
Transfer.
XML.
JSON

Flipkart

Amazon

website

web applications

XML JSON

HTML

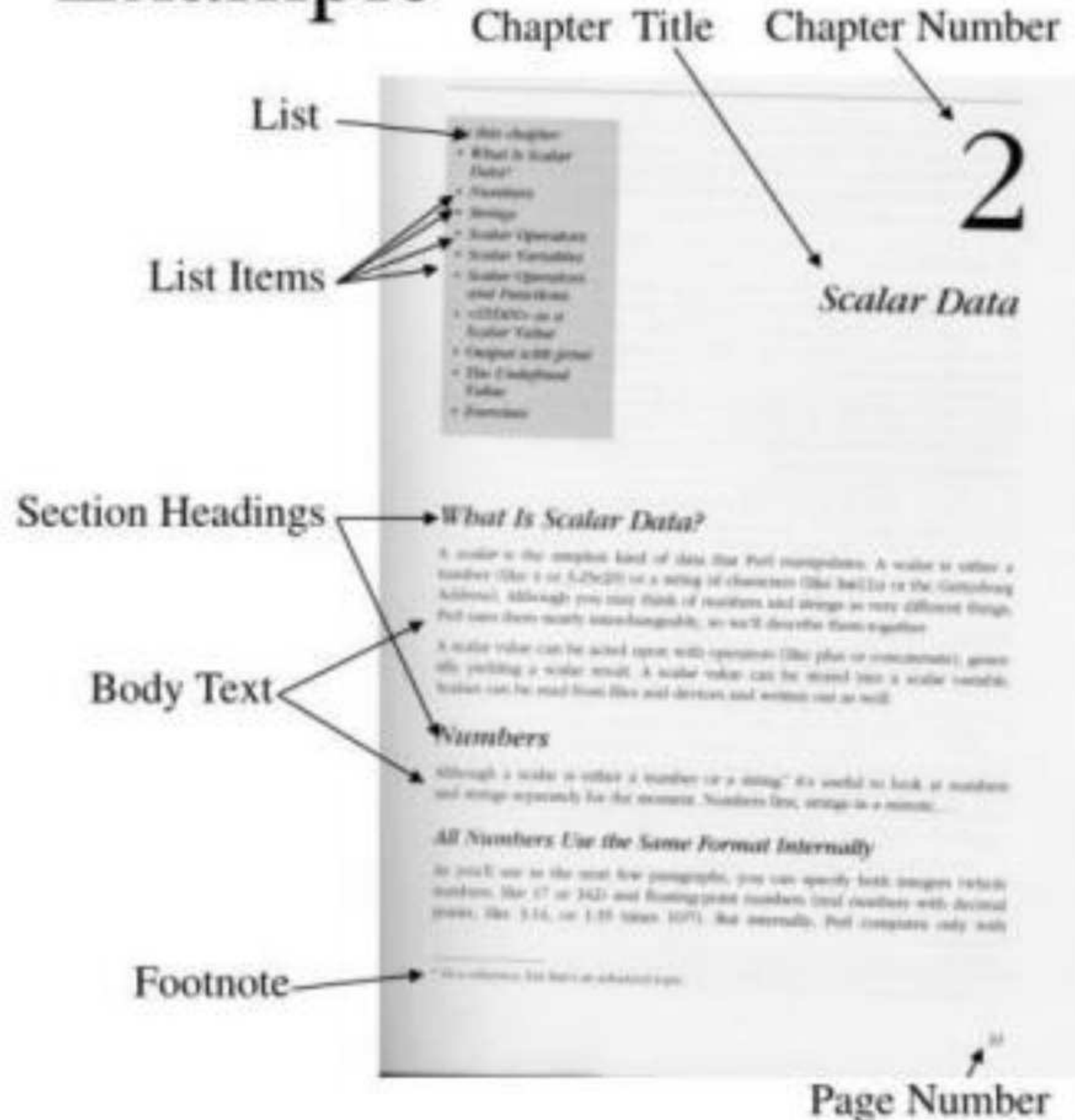
- What does HTML stand for?
 - HyperText Markup Language
 - HTML documents are simply text documents with a specific form.
- What does “markup” mean?
 - Comes from publishing.
 - Documents comprised of content and markup.
 - Content: actual information being conveyed.
 - Markup: side information about the content.

Markup

- Markup information is used to convey the role of different parts of the content and their relationships to each other.

Example

- Here is the first page of chapter two from “Learning Perl”.
- What are the roles of the different pieces of content?



HTML Tags

- Markup is indicated through the use of “tags”.
 - Tags are delimited by angle brackets `<>`
 - Beginning tags
 - Can also include attributes.
 - `<tagname a1=“v1” a2=“v2”>`
 - Ending tags
 - `</tagname>`
 - No attributes.

HTML Document Structure

- Entire document enclosed within `<html>` and `</html>` tags.
- Two subparts parts:
 - Head
 - Enclosed within `<head>` `</head>` tags.
 - Within the head further tags can be used to specify title of page, meta-information, etc.
 - Body
 - Enclosed within `<body>` `</body>` tags.
 - Within the body is the document content to be displayed.

Basic Tag Types

- `<title>`
 - In the head of the document, provides a title for the page.
- `<h1>`, `<h2>`, `<h3>`
 - Different heading levels.
- `<p>`
 - Indicates paragraphs.
- `<br>`
 - Indicates line breaks.

Tag Attributes

- Attributes specified in beginning tag modify and amend tag meaning.
- Each tag has a predefined set of meaningful attributes.
- Unrecognized attributes are ignored.
- Example:
 - Source file for image data.
 - ``

Anchors and Links

- Key feature of HTML is “linking” between documents.
- Portion of content can be specified as a link using the anchor tag `<a>`
- The href attribute of the anchor tag provides the address of the linked document.
- Example:
 - `<a href=“http://www.cs.unc.edu/~kmp”>`

Anchors

- The target of a link can further specify a specific “anchor” to go to.
 - href=http://server/path/to/page#anchorname
- Defining anchors.
 - Name attribute of an <a> tag.
 -
 - ID attribution of any other tag.
 - <h1 id=“anchorname”>

Embedded Objects

- Other types of data can be embedded into an HTML document.
 - Most commonly: images
- Images indicated using the `<img>` tag.
- The `src` attribute indicates where the image data can be found.
- Example:
 - ``

Function Not Format

- Markup the function of the content without specifying format of the content.
 - Allows end user to decide how function was mapped to appearance.
 - Example: `<em>` for emphasis.
- What are the advantages of doing this?
- What are the disadvantages?

Problems with HTML

- HTML became “polluted” with formatting specific tags and tag attributes.
 - Example: `<b>` for bold.
- Different companies kept adding “custom” tags.
 - Why is this a problem?
- Although human-readable, not always easy for machine to parse and manipulate.
 - What could be hard for the machine but easy for the human?
 - Why might we want a more machine friendly document?

XML to the rescue

- Document format for structured data.
- Extensible markup languages
 - Domain-specific
 - Common agreements and standards.
- Separate form, meaning, and presentation.
 - Form defined by XML
 - Meaning defined by applications and XML-based standards.
 - Presentation defined separately from content.

XML Document Structure

- XML Declaration
 - Required
- Document Type Declaration
 - Optional
- Document Body
 - Required

XML Declaration

- `<?xml version="1.0" encoding="UTF-8" ?>`
 - Must be first line of the document.
 - The version attribute is required.
 - Two optional attributes:
 - encoding
 - Standard name for character set encoding.
 - UTF-8, UTF-16, US-ASCII, etc.
 - standalone
 - “yes” or “no”
 - Indicates whether or not this XML document requires an external document type definition.
 - Default is “no”
 - Not required to be set.

XML Declaration

- Valid declarations:

- `<?xml version="1.0" ?>`

- `<?xml version="1.0" encoding="UTF-16" ?>`

- `<?xml version="1.0" standalone="no" encoding="UTF-8" ?>`

- `<?xml version="1.0" encoding="US-ASCII" standalone="yes" ?>`

- Invalid declarations:

- `<xml version="1.0">`

- `<?xml version="1.0">`

- `<?XML Version="1.0"?>`

- `<?xml standalone="no" version="1.0"?>`

- `<?xml version=1.0" ?>`

DTD Declaration

- Two forms:

- <!DOCTYPE *RootElement* SYSTEM *DTD_URI*?
[*InternalDeclarations*]? >

- <!DOCTYPE *RootElement* PUBLIC *PUBLIC_ID URI*
[*InternalDeclarations*]? >

- Here “?” indicates optional.

- DTD specifies how the XML document can be validated.

Document Body

- The document body is comprised of markup and character data.
- Must contain at least one tag that encloses everything.
 - This is called the root element.

Anurag, Gautam, and Saifraz are online learners
and Sweety is a female online learner.

<learners>

<online learner>

<name> Anurag </name>

<name> Gautam </name>

<name> Saifraz </name>

</online learner>

<online female learner>

<name> Sweety </name>

</online female learner>

</learners>

Markup vs. Character Data

- Markup is text in specific formats that defines the structure of the document and provides interpretation and parsing hints.
- Character data is everything else.

Markup

- XML declaration
- Document type declaration
- Element Tags
 - Start, End, and Empty
- Entity References
 - External, internal, and character.
- Comments
- CDATA
- Processing Instructions

XML Declaration and DTD

- Already reviewed what these markup elements look like.
- We'll get into DTD later.

Elements

- Elements are defined by a start and end tags.
 - Start tags: `<element_name attributes*>`
 - End tags: `</element_name>`
- Element Names:
 - Case sensitive
 - Can start with letter, underscore or colon
 - Can contain letters, underscore, colon, digits, hyphens, or periods.
- Everything between start and end tags is considered element content.

Attributes

- Form

attribute_name=“*value*”

attribute_name=‘*value*’

- Attribute names have same rules as element names and are also case sensitive.
- Values must be quoted
 - Can use either single- or double-quotes.

Reserved Attributes

- Some attributes are given special meaning and can be used with any element.
 - xml:lang
 - Provides language/character set information for everything within this element.
 - xml:space
 - Provides hint to parser whether whitespace is really significant or not.
 - Value can be “default” or “preserve”
 - Unfortunately, “preserve” is the default?!?!
 - xml:base
 - Controls base URI for document for resolving relative links and references.
 - xmlns
 - Used to define namespaces.

Empty Elements

- Sometimes elements are never intended to have content.
 - Think of HTML `<br>` tag.
 - If we wanted to make HTML XML-compliant:
 - `<br></br>`
 - Empty tag notation is shorthand for this:
 - `<element_name attributes*/>`
 - Example: `<br/>`

Comments

- Comments delimited by:
 - Start of comment: `<!--`
 - End of comment: `-->`
- Comment content ignored.
 - No meaning as markup or data.
- Restrictions
 - Cannot contain “--” except as part of delimiters.
 - Cannot be nested.
 - Cannot precede the XML declaration.
 - Cannot be within element tag (start, end, or empty)

Processing Instructions

- Also known as PI's
- Form:
 - `<?pi_name pi_data?>`
- Restrictions
 - No space between ? and *pi_name*
 - *pi_data* cannot contain “?>”
 - What *pi_data* means is defined externally.
- XML declaration is actually an example of a PI.
 - Some *pi_names* defined by specific standards.

Entities

- Given what we've seen so far, can you think of element content text that might cause problems?
 - What if the character data (i.e., element content) contains “<”?
 - Would be difficult to tell whether or not this was the start of a new tag or not.
 - Example:
`<equation> 3 < x + 2 </equation>`

Entities

- Entities provide a way to specify text which would be hard to specify directly.
 - “<” or “>” for example.
 - Unprintable characters.
 - Unicode characters for other languages.
- An XML parser interprets an entity reference and replaces it with its definition.
- Form:
&entity_name;

Predefined Entity References

- XML standard defines a number of entities references that all XML parsers should support:
 - < for “<”
 - > for “>”
 - & for “&”
 - " for double quote
 - ' for single quote

Using Entities

- Required to use entity references for “<” and “&” except within comments, PI’s, or CDATA.
- Can also use entities to refer to character by its encoding value:
 - &#dec_char_value;*
 - &#xhex_char_value;*
- Useful for non-printable and foreign characters.
- Replacement text is reparsed.

CDATA

- Good for large sections of text which may be difficult to make adhere to the rules.
 - For example, suppose part of the document includes programming code.
- CDATA sections indicates that everything in it is character data without parsing and interpreting.
 - Starts with: `<![CDATA[`
 - Ends with: `]]>`
 - Only restriction is that it can't contain `“]]>”`

CDATA Example

<code-fragment>

<![CDATA[

Integer[] foo(a, b) {

Integer[] result = new Integer[(b-a)];

for (i = a; i < b; i++) {

result[i] = bar(i, a, b);

}

return result;

}

]]>

</code-fragment>

Well Formed

- XML provides structure not meaning.
- A document is XML if it is “well formed”
- Rules for well formed:
 - Must start with xml declaration.
 - Tags must nest perfectly.
 - Good: `<a> <b> </b> </a>`
 - Bad: `<a> <b> </a> </b>`
 - Bad: `<br>`
 - If there is no content, then must use empty tag form: `<br/>`
 - All attribute values must be quoted.
 - All elements must be enclosed by a single root tag.
 - Elements form a tree.

Sharing Vocabularies

- Suppose I were to invent a set of XML tags for my bookstore. What would they be?
 - `<book>`
 - `<title>`
 - `<author>`
 - `<price>`
 - Attributes for the tags to add descriptive power.
 - `<book genre="Mystery">`
 - Organization and meaning decided by me.
- What problems arise if I publish or distribute my XML files which describe my books?

Sharing XML Problems

- P1: Meaning of a tags are specific to whomever invented them.
- P2: Different people may invent the same tag names.
 - Library of Congress: `<book>`
 - My Bookstore: `<book>`
 - Amazon: `<book>`
 - HTML: `<title>`
- Solutions?

Sharing XML Solutions

- Answer to P1, is standardization and documentation.
 - Create a standardized description for what a set of tags means and how they are to be used.
 - This is the alphabet soup of the XML big picture.
 - Want as much agreement as possible
 - But you can always strike out on your own.
 - Or, develop tags for internal use.
 - Still have problem of different tag sets using the same names.
- Answer to P2 is “namespaces”

Namespaces

- Motivation:
 - “Define a mechanism for uniquely naming elements and attributes so different vocabularies can be mixed into an XML document without name conflicts.”
 - Sall, “XML Family of Specifications”, p. 211.

Namespace Overview

- Every tag vocabulary is identified by a URI.
 - This URI doesn't actually have to point to anything.
 - Could point to the documentation of your XML
 - Could point to the organization that created the standard
 - Or it could be something you just made up!
 - It is just a way to come up with a single, public, well-known name for the vocabulary.
- Associate a prefix with the namespace.
- Use qualified element and attribute names to link name with namespace.

Declaring a Namespace

- xmlns attribute
 - Form: `xmlns:prefix="URI"`
 - `<kmp:bookstore xmlns:kmp="http://bookstore.example.com/foo/bar">`
 - Prefix associates tags and attributes with this namespace.
 - Remember: the URI identifies the namespace.
 - The prefix can be almost anything.
 - Should not be "xml"
 - Applies only to this document.
 - Example:
 - `<html:title> My Bookstore </html:title>`
 - `<book:title> The Art of Computer Programming </book:title>`

Namespace scope

- Scope of prefix is the element in which the xmlns attribute is given.
 - Allows you to limit use to just where you need it.
 - Applies immediately (i.e., including the element declaring the namespace).

`<foo:tag xmlns:foo="http://bar/baz">`

(scope of the "foo" prefix)

`<foo:blah>` This is content of the blah tag. Presumably blah means something in the set of tags associated with the URI `http://bar/baz`.

`</foo:blah>`

`</foo:tag>` *(end of "foo" prefix scope)*

`<foo:blah>` While still legal syntax, this content can not be interpreted with respect to the tags associated with `http://bar/baz`

`<foo:blah>`

Qualified Names

- Qualified element and attribute name form:
 - *prefix:name*
- Prefix was given in namespace declaration.
- Name is element or attribute name defined within that namespace.
- Examples of qualified names:
 - xhtml:ul
 - xlink:href
 - kmp:book
 - foo:bar

Default Namespace

- Useful if most/all tags are part of the same namespace.

- Form:

– `xmlns="URI"`

Everything until the closing `</html>` tag will be in this namespace by default

- Example:

• `<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">`

This lang attribute uses the 'xml' prefix

This lang attribute uses the default xhtml namespace

Namespaces in XHTML

- Official URI identifying XHTML tag vocabulary:
 - <http://www.w3.org/1999/xhtml>

But what does it mean?

- Like mama used to say, “XML is as XML does.”
 - It’s just a way of storing structured information.
- Then why bother?
 - Can define separate specifications which do attach meaning to specific tag “vocabularies”.
 - Can build software tools and libraries which manipulate XML data without having to know about its meaning.
 - These components are then reusable and can be part of applications which do attach meaning.
 - Can define validity in ways that can be verified by machine.

Validity

- XML 1.0 specifies form.
- Doesn't specify meaning.
 - One aspect of meaning is validity.
 - What does validity mean to you?
- In XML, validity is defined in terms of “correct” form.
 - How is this different from “well” formed?
- Need a way to define what “correct” form is.

DTD

- Document Type Definition
 - A formal set of rules that define “correct” form.
- Why is this useful?
 - Can serve as a standard that multiple organizations can agree to.
 - XML only useful if useful vocabularies are developed.
 - Allow automatic validation.
 - This is the difference between validating and non-validating XML parsers.
 - Specify default values for attributes.
 - Defines entities that may be used.

Declaring DTD's

- XML declares which DTD is in effect.
- Remember structure of XML document:
 - XML Declaration
 - Required
 - Document Type Declaration
 - Optional
 - Document Body
 - Required

DOCTYPE

- DTD declaration in one of two forms:
 - <!DOCTYPE *RootElement* SYSTEM *DTD_URI*? [*InternalDeclarations*] ?>
 - <!DOCTYPE *RootElement* PUBLIC *PUBLIC_ID URI* [*InternalDeclarations*] ? >
- First form is used when DTD is “local” to the current document.
 - By local, we mean the XML document and the DTD were both created by the same person/organization/community.
 - The other form is used for a more widely known and agreed upon DTD.

SYSTEM DTD

`<!DOCTYPE RootElement SYSTEM DTD_URI? [InternalDeclarations]? >`

- RootElement

- Definition of this element is expected to be in the DTD

- DTD_URI is location of DTD document

- Can be relative or absolute

- InternalDeclarations

- Can amend or add on to a DTD with additional info.

- Examples:

`<!DOCTYPE BookList SYSTEM "bookstore.dtd">`

`<!DOCTYPE BookList SYSTEM`

`"http://wwwj.cs.unc.edu:8080/kmp/bookstore.dtd">`

PUBLIC DTD

<!DOCTYPE *RootElement* PUBLIC *PUBLIC_ID* *URI* [*InternalDeclarations*] ? >

- This form used for widely-known, publicly agreed upon DTD's.
 - Defines validity for standard vocabularies expected to be used by many people/organizations.
- PUBLIC_ID
 - Widely known and standardized name for the DTD.
- URI
 - Location of a particular copy of the DTD
 - Why might we want to have many copies?

PUBLIC DTD Names

- There is a convention for what the PUBLIC_ID looks like.
- Examples:
"-//NASA//Instrument Markup Language 0.2//EN"
"-//W3C//DTD XHTML Basic 1.0//EN"

What a DTD can do for you

- Specify elements and their attributes.
- Nesting of elements
- Order of elements
- Declare whether an element contains
 - nothing
 - non-markup character data
 - mix of character data and markup
- Whether elements are optional or required.
- Whether attributes are optional or required.
- Default and fixed values for attributes.
- Define entities.

Validation

- Given XML document and associated DTD definition, can automatically validate the document against the DTD.
 - Parse the DTD to understand the rules.
 - Parse the XML and make sure the rules fit.
 - A well formed XML document may not be valid.
 - Well-formedness is defined by rules of XML for all XML docs.
 - Validity is specified by specific DTD for a specific use of XML.

Other Ways To Validate

- DTD is just one mechanism to validate.
 - Has its strengths and weaknesses
 - + In-line definition available.
 - + Simple syntax.
 - - Simple syntax.
- Other schemes:
 - XML-Schema
 - RELAX NG
 - Schematron
- For more see in Wikipedia:
 - XML Schema Language Comparison

HTML 4.01 (XHTML 1.0)

- HTML now an application of XML
 - XHTML 1.0 == HTML 4.01
 - Deprecated format specific tags and attributes.
 - Defined DTD's for various levels of conformity.
 - Now machine verifiable.
- Introduced cascading style sheets (CSS).
 - W3C recognized need to specify format as well.
 - Formalizes how presentation and formatting is specified.
 - Allows many documents to use common stylesheet

XHTML DTD's

- XHTML 1.0 Strict

```
<!DOCTYPE html PUBLIC  
"-//W3C//DTD XHTML 1.0 Strict//EN"  
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

- XHTML 1.0 Transitional

```
<!DOCTYPE html PUBLIC  
"-//W3C//DTD XHTML 1.0 Transitional//EN"  
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

- XHTML 1.0 Frameset

```
<!DOCTYPE html PUBLIC  
"-//W3C//DTD XHTML 1.0 Frameset//EN"  
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-frameset.dtd">
```

Important XHTML Rules

- Must be well-formed XML.
 - Perfect nesting.
 - Quoted attribute values.
 - All elements must be closed.
- Must specify one of the three XHTML DTD's as the document type.
- Tags must come from the XHTML namespace.
 - Element names must be lowercase.
 - Small but important difference with old HTML

Summary

- HTML as the lingua franca of web documents.
 - The concept of a markup language
 - Annotating document content with role information.
- Motivating XML
 - Extensible framework for structured data.
 - Tree of elements decorated with attributes
 - Separating content, format, and meaning.
- XML details.
 - Rules of well-formedness
 - Well-formed vs. validity.
 - DTD as a mechanism to specify validity.
- XHTML as HTML done right (or at least better).



2 mins Summary



TEACHING
WALLAH

Topic

Client Server Architecture

API

Different Layers

Front End Html

XML, Js

THANK YOU